詳細!PHP8+MySQL 入門ノート 訂正補足

Ver.4.1.3

Seiichi Nukayama

2026 年 4 月 7 日

目次

1 PHP の準備 1
1.1 p21 — xampp-control.ini のアクセス権の変更 1
1.1.1 XAMPP コントロールパネルを 管理者権限で起動できるようにする 1
1.1.2 MySQL(MariaDB) をサービスに登録する 2
1.2 p27 — php.ini 3
1.2.1 php.ini の設定 3
1.2.2 my.ini の設定 3
1.3 VSCode で PHP を書くための設定 5

2 php を動作させる 3 つの方法 6
2.1 php を動作させる 3 つの方法 6
2.2 (1) php コマンドで動作させる 6
2.3 (2) Apache サーバーで動作させる 7
2.4 (3) PHP 簡易サーバー (Built-in Web Server) を使う 8

3 URL エンコード 10
3.1 p164, p165, p303, p304, p305 — URL エンコード 10
3.1.1 urlencode() と rawurlencode() 10
3.1.2 URL エンコードしてみる 11
3.1.3 URL デコードは必要か? — 必要ない 11

4 エンコード・チェック 13
4.1 p310 — cken() 関数 13

5 HTML エスケープ 14
5.1 著者の提示している方法 14
5.2 HTML エスケープをどのタイミングでおこなうか? 20
5.2.1 二重エスケープはまずい 20
5.2.2 HTML エスケープは画面出力のタイミングで 21
5.3 p308 — es() 関数 21

6 PDO::PARAM_XXX のはたらき 22
6.1 p559 — PDO::PARAM_XXX 22
6.1.1 PDO::ATTR_EMULATE_PREPARES, false の場合 22
6.1.2 PDO::PARAM_INT の働き 22

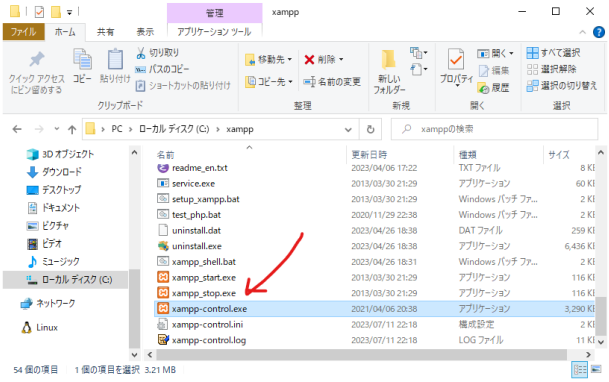
1　PHP の準備

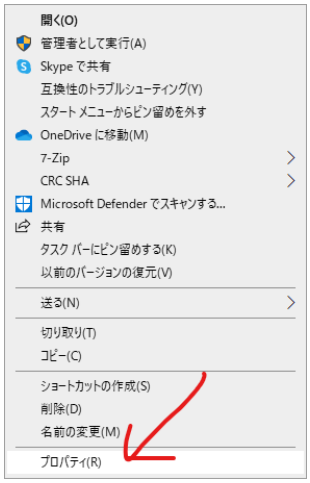
1.1　p21 — xampp-control.ini のアクセス権の変更

1.1.1　XAMPP コントロールパネルを 管理者権限で起動できるようにする

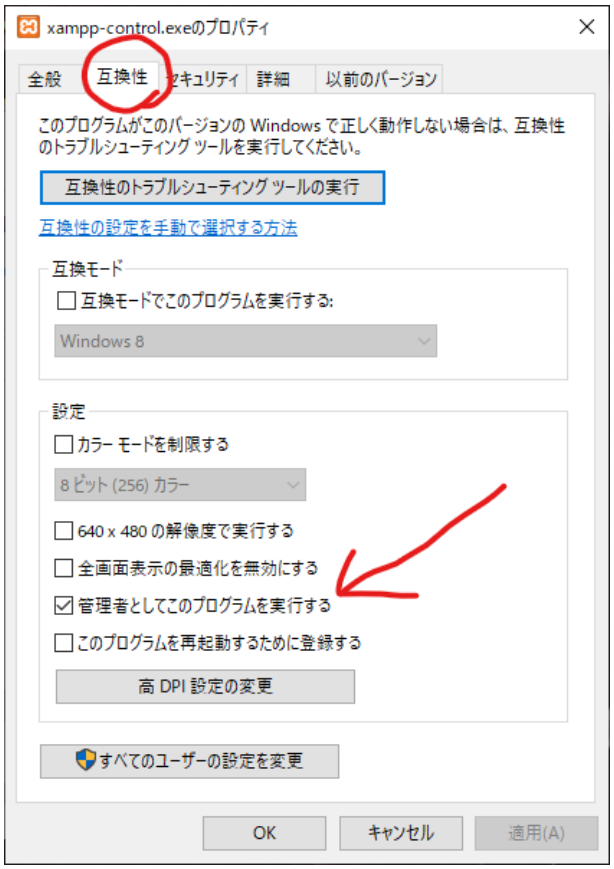
ここでは、“xampp-control.ini” のアクセス権を変更しているが、それよりも “C:¥xampp” フォルダの “xampp-control.exe” を管理者権限で実行できるようにした方がよい。

“C:¥xampp” フォルダを開く。“xampp-control.exe” を右クリックして、“プロパティ” を選択する。





開いた プロパティ・ウィンドウで、“互換性” タブを開き、“ 管理者としてこのプログラムを実行する” にチェックを入れる。



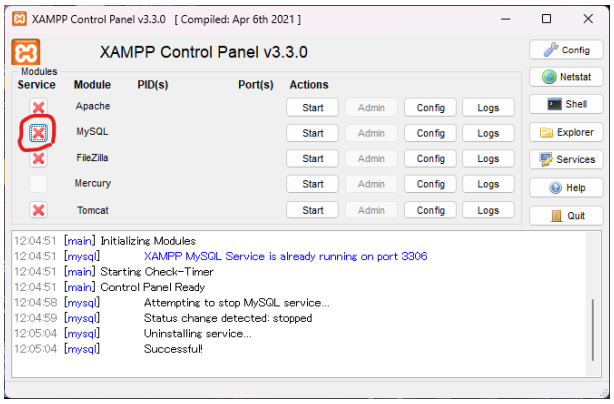
これで、XAMPP-Control パネルは常に 管理者権限で起動される。

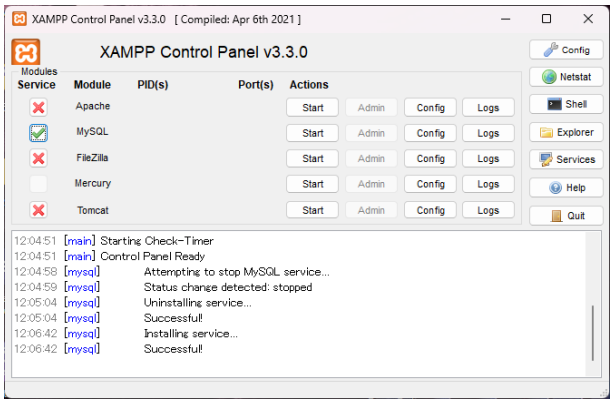
1.1.2　MySQL(MariaDB) をサービスに登録する

MySQL を起動させた場合、MySQL を停止させずに Windows を終了させると、MySQL のなかのテーブルが破損され、次回 MySQL が起動できなくなることがたまに起こる。

これは、MySQL をサービスに登録することで解決する。Windows は終了時に、サービスに登録されているアプリケーションを停止させるからである。

MySQL をサービスに登録するには、一番左の “Module Service” の × 印のボタンをクリックする。この時、MySQL が動作していないことを確認すること。動作中ではサービスに登録できない。





MySQL をサービスに登録できたら、Apache も同様にサービスに登録したほうがいいだろう。

1.2 p27 — php.ini

1.2.1 php.ini の設定

php.ini での設定は、

2010 data.timezone=Asia/Tokyo

だけでよい。

いろいろと細かく書かれてあるが、これらは PHP5.6 以前 (だったと思う) の設定である。PHP5.6 以後は、ここで書かれてある設定は" 非推奨" となっている。この本の著者がこの設定の記述を改めないのは何故なのか、不思議である。(本屋さんで、ほかの本の記述を確認してみてほしい)

> mbstring の設定は" 非推奨" である。
> (参考) PHP マニュアル — mbstring — 実行時設定

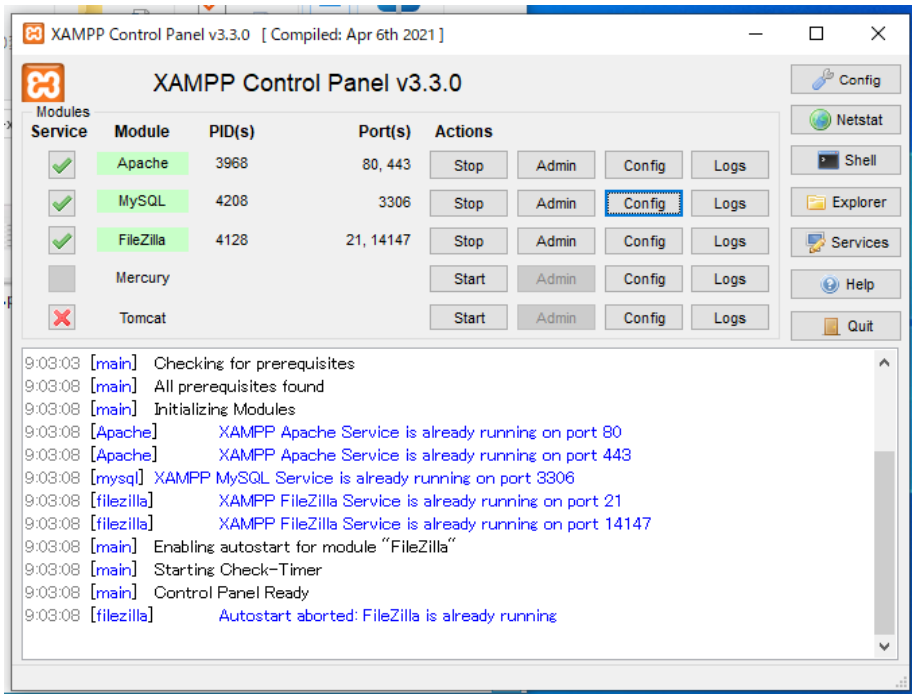
これは、p38、p47 も同様である。

ただ、WordPress のバックアップのことを考慮すると、以下の設定もしておいてよい。

434	memory_limit=1024M
703	post_max_size=512M
855	upload_max_filesize=512M

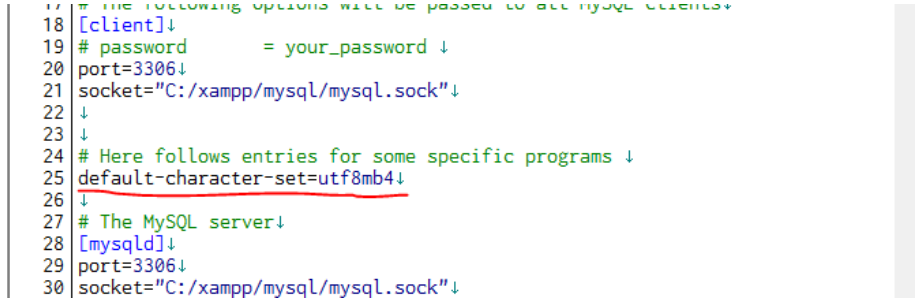
1.2.2 my.ini の設定

MySQL の設定は、おそらく以下のようになっているはずなので、確認しておく。XAMPP コントロールパネルの MySQL の "config" をクリックする。



“config” をクリックして “my.ini” を選択すると、“my.ini” が表示される。
“[client]” の領域に 次の 1 行があるはずだが、もし無ければ追加する。

```
default-character-set=utf8mb4
```



次に “[mysqld]” の領域に以下の 2 行があるはずだが、もし無ければ追加する。

```
character-set-server=utf8mb4
collation-server-utf8mb4_general_ci
```

```
157 sql_mode=NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTION
158 log_bin_trust_function_creators=1
159
160 character-set-server=utf8mb4
161 collation-server=utf8mb4_general_ci
162
163 [mysqldump]
164 max_allowed_packet=16M
165
```

設定ができたら、MySQL を再起動する。その後、コマンドプロンプトで、MySQL に root でログインし、次のコマンドを実行する。

```
MariaDB [(none)]> show variables like '%char%';
```

次のように表示されればよい。

```
MariaDB [(none)]> show variables like '%char%';
+--------------------------+--------------------------------+
| Variable_name            | Value                          |
+--------------------------+--------------------------------+
| character_set_client     | cp932                          |
| character_set_connection | cp932                          |
| character_set_database   | utf8mb4                        |
| character_set_filesystem | binary                         |
| character_set_results    | cp932                          |
| character_set_server     | utf8mb4                        |
| character_set_system     | utf8                           |
| character_sets_dir       | C:\xampp\mysql\share\charsets\ |
+--------------------------+--------------------------------+
8 rows in set (0.001 sec)
```

character_set_database と character_set_server が utf8mb4 になっていればよい。

1.3 VSCode で PHP を書くための設定

1. PHP へのパスを Windows のシステム環境変数の"Path" に登録しておく。(C:¥xampp¥php)
   (a) "システム環境変数の編集" を検索して開く。
   (b) "環境変数" をクリック。
   (c) "Path" を選択して "編集"
   (d) "新規" を選択して"C:¥xampp¥php" を入力。

2. VSCode の拡張機能として、"PHP Server" をインストールする。

3. VSCode の拡張機能として、"PHP Intelephense" をインストールする。
   "Quick Start" に書かれてあること (1 番) を実行する。
   (a) 拡張機能を開く。
   (b) "@builtin php" で検索。
   (c) "PHP 言語機能" を無効化する。(PHP の基本言語サポートは有効のまま)

2　php を動作させる 3 つの方法

2.1　php を動作させる 3 つの方法

拡張子が php であるファイルに書かれた php コードを動作させるには、3 つの方法がある。

1. php コマンドの引数に php ファイルを指定して動作させる。
2. C:¥xampp¥htdocs フォルダに php ファイルを配置する。
3. php 簡易サーバー (ビルトイン・ウェブサーバー) を使う。

2.2　(1) php コマンドで動作させる

php ファイルを次のように用意する。“c:¥Users¥user¥Desktop¥LaLa¥10_php¥php_work” に “test” フォルダを作成し、“test.php” を作成する。

test.php

```php
<?php
echo "This is TEST!";
```

そのフォルダでターミナルを開き、次のようにする。

```
> php test.php
```

実行例

```
This is TEST!
```

同じフォルダに次のようなファイルを作成する。

sample.php

```php
<?php
$msg = "This is TEST!";
?>
<!doctype html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>TEST</title>
</head>
<body>
```

```
11    <h1><?php echo $msg; ?></h1>
12  </body>
13  </html>
```

実行する。

```
> php sample.php
```

次のように<?php ... ?>の部分が実行され、表示される。

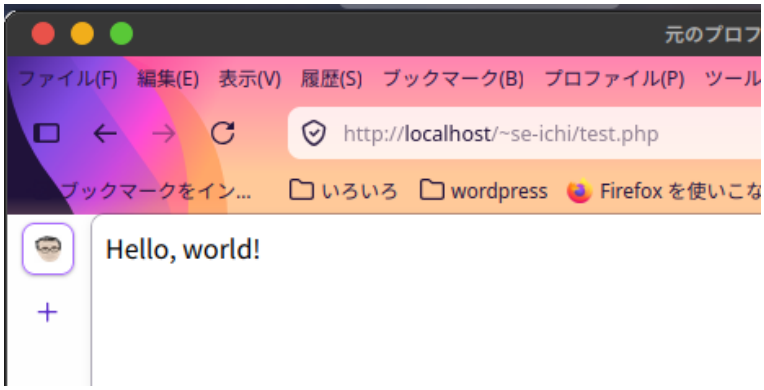
実行例

```
<!doctype html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>TEST</title>
</head>
<body>
  <h1>This is TEST!</h1>
</body>
</html>
```

この方法はそんなに多用しない。主にデバッグのために使うことになる。

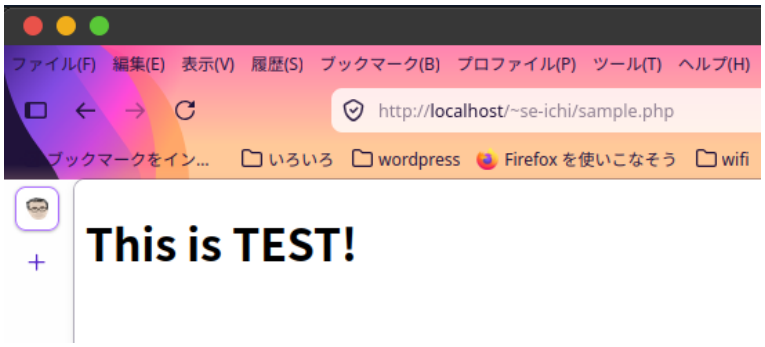
2.3 (2) Apache サーバーで動作させる

先ほどの test.php を “C:¥xampp¥htdocs” に置く。
ブラウザで、“http://localhost/test.php” と URL を指定すると、



また、先ほどの sample.php を “C:¥xampp¥htdocs” に置く。

ブラウザの URL で、“http://localhost/sample.php” と指定すると、ブラウザに This is TEST が表示される。



この方法が、PHP に期待されている使われ方である。Apache サーバーとの連携で動作している。

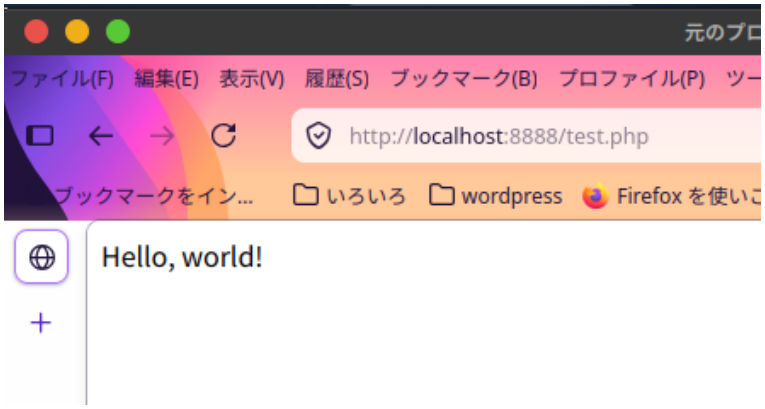
2.4 (3) PHP 簡易サーバー (Built-in Web Server) を使う

“c:¥Users¥user¥Desktop¥LaLa¥10_php¥php_work¥test” フォルダでターミナルを起動する。
ここで、次のように、PHP 簡易サーバー (ビルトイン・ウェブサーバー) を起動する。

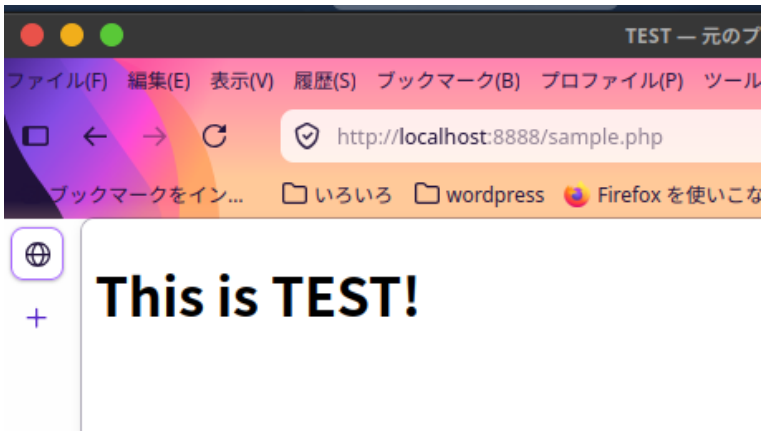
```
> php -S localhost:8888
```

8888 — ポート番号は任意

“http://localhost:8888/test.php” にアクセスすると、



と表示される。
“http://localhost:8888/sample.php” にアクセスすると、



と表示される。

この簡易サーバーは便利だが、いちいちコマンドを入力するのがわずらわしい。この機能を使ったのが、VSCode の拡張機能である “PHP サーバー” である。この簡易サーバーの起動と、ブラウザでの表示をワンクリックでできる。

この方法は、PHP プログラムを単体で動作させ、その動きをみたり、デバッグするのに使われる。

3　URL エンコード

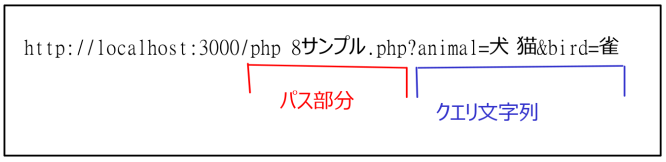
著者は、パス文字列とクエリ文字列、urlencode() と rawurlencode() で混乱しているので、著者の記述を読むと読者の頭も混乱してくるので、この部分は本は読まないほうがよい。

3.1　p164, p165, p303, p304, p305 — URL エンコード

3.1.1　urlencode() と rawurlencode()

URL 文字列に、制御文字や空白文字、マルチバイト文字 (全角文字) などが使われている場合は、URL エンコードが必要になる。

URL エンコードには、urlencode() 関数と rawurlencode() 関数がある。以下のように使い分ける。



パス部分に制御文字や空白文字、マルチバイト文字が含まれている場合 → rawurlencode() を使って URL エンコードする。

クエリ文字列に制御文字や空白文字、マルチバイト文字が含まれている場合 → urlencode() を使って URL エンコードする。

両方に制御文字や空白文字、マルチバイト文字が含まれている場合は、全部に rawurlencode() を使って URL エンコードする。

> Web サーバーは、rawurlencode されたパス文字列をデコードして処理をする。この時、urlencode されたパス文字列だと、エラーとなる。
> PHP は、urlencode されたクエリ文字列をデコードして、$_GET にセットする。rawurlencode されたクエリ文字列であっても、自動でデコードしてくれる。

> urlencode とは、RFC1738 で定められたエンコードで、『-(ハイフン)_(下線).(ピリオド)(~チルダ) を除くすべての非英数文字が % 記号に続く二桁の 16 進数で置き換えられ、空白は + 記号にエンコードされます。同様の方法で、WWW のフォームからポストされたデータはエンコードされ、`application/x-www-form-urlencoded` メディア型も同様です。歴史的な理由により、この関数は RFC3986 エンコード (rawurlencode() を参照ください) とは異なり、空白を + 記号にエンコードします。』
>
> (PHP マニュアル | urlencode より引用)

3.1.2 URL エンコードしてみる

サンプルコードは、次のようになる。

index.php

```
<?php
  $url = rawurlencode("php 8 サンプル.php");
  echo $url;   // php%208%E3%82%B5%E3%83%B3%E3%83%97%E3%83%AB.php

  $animal = urlencode("犬 猫");
  $bird = urlencode("雀");
  $query = "animal={$animal}&bird={$bird}");
  echo $query;  // animal=%E7%8A%AC+%E7%8C%AB&bird=%E9%9B%80
?>
<a href="<?= $url ?>?<?= $query ?>">リンク</a>
```

最後の行は、次のようにも書ける。

```
<?php
  echo "<a href=\"{$url}?{$query}\">リンク</a>";
?>
```

受け取るコードは次のようになる。

php 8 サンプル.php

```
<?php
  $animal = $_GET['animal'];
  $bird = $_GET['bird'];
  echo $animal;  // 犬 猫
  echo $bird;    // 雀
?>
```

3.1.3 URL デコードは必要か? — 必要ない

パス部分の場合は、Web サーバーが受け取ってデコードするので、プログラマが介在するタイミングはない。

クエリ文字列の場合は、PHP がすでにデコードして $_GET にセットしてくれているので必要ない。(rawurlenoce であっても、urlencode であっても)

警告 スーパーグローバル $_GET および $_REQUEST の内容はすでにデコード済みのものです。$_GET や $_REQUEST の要素に対して urldecode( ) を使用すると、予期せぬ危険な結果となります。

引用: urldecode | PHP マニュアル

4　エンコード・チェック

4.1　p310 — cken() 関数

p310 の cken() 関数は、mb_check_encoding() が文字列しか扱えなかった時代のコードである。配列ならそれを文字列にして、mb_check_encoding() に渡している。

mb_check_encoding() 関数は、文字列だけではなく、配列を再帰的に扱うことができるようになったので、p310 の cken() 関数は、次のように書くことができる。

lib_cken/lib/util.php

```
function cken($data) {
  return mb_check_encoding($data, 'UTF-8');
}
```

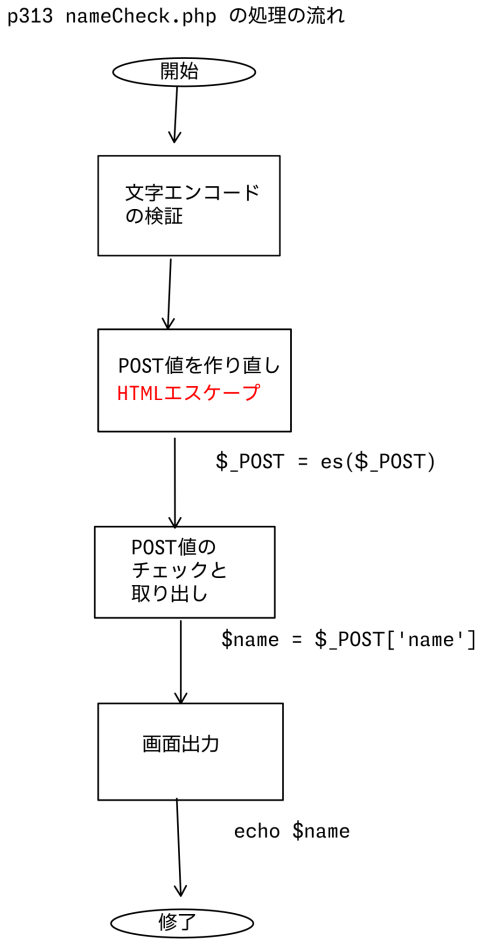
第 2 引数は" 期待する文字エンコード" で、省略すると、mb_internal_encoding() 関数と同じになる。ここでは、明示しておいた。

5　HTML エスケープ

5.1　著者の提示している方法

著者が HTML エスケープ処理を行っているタイミングを p.313 の nameCheck.php を例にとって見てみる。

ふつうの処理の場合



このように、第 8 章、第 9 章のサンプルコードでは、フォームから送られてきたデータを処理する場合、

1. 文字エンコードの検証
2. POST 値の作り直し (HTML エスケープ)
3. POST 値のチェックと変数への取り出し
4. 変数の画面出力

というパターンで行なわれている。

つまり、(1) 文字エンコードのチェック (2) HTML エスケープという順序で処理をすることを著者は提示している。

そうなると、初心者は、次のようにすることになる。

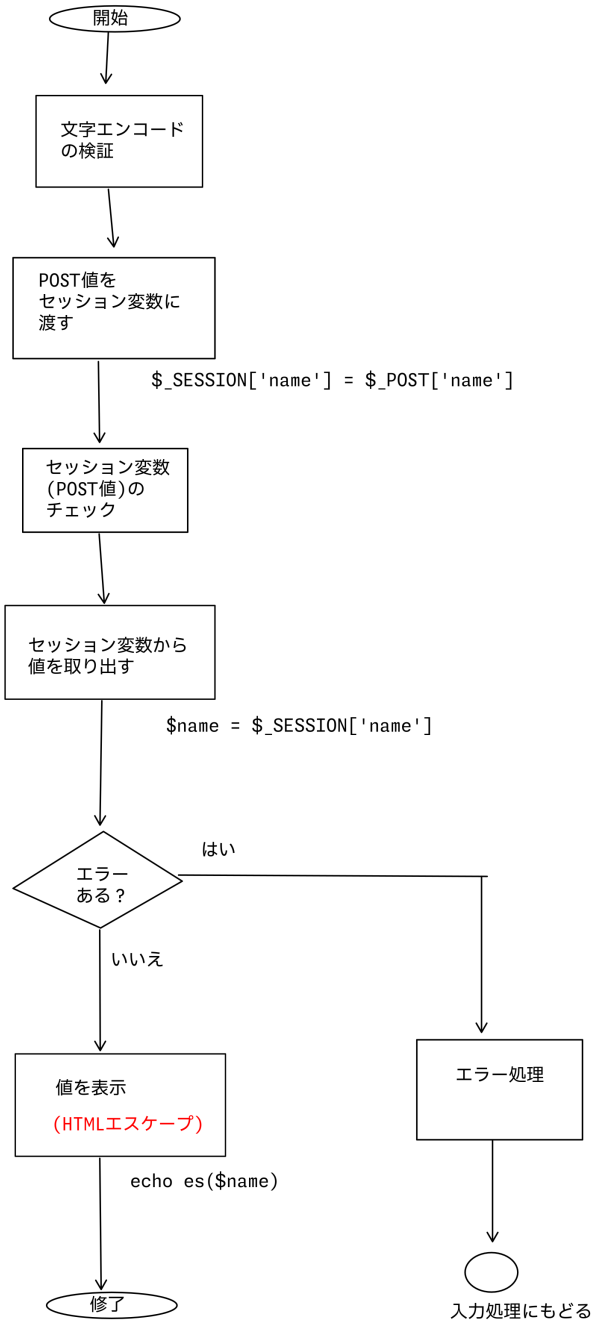
1. util.php を作成して、そこに es() 関数 をコピーして貼り付ける。
2. まず、文字エンコードのチェックをする。
3. 次に HTML エスケープをする。
4. 入力値のチェックをする。
5. 画面出力をする。

以上の処理を定型的に記述することになるだろう。

しかし、たとえば第 10 章" セッション" のところでは、次のような順序で勝利をしている。

セッション処理の場合

p419 confirm.php の処理の流れ



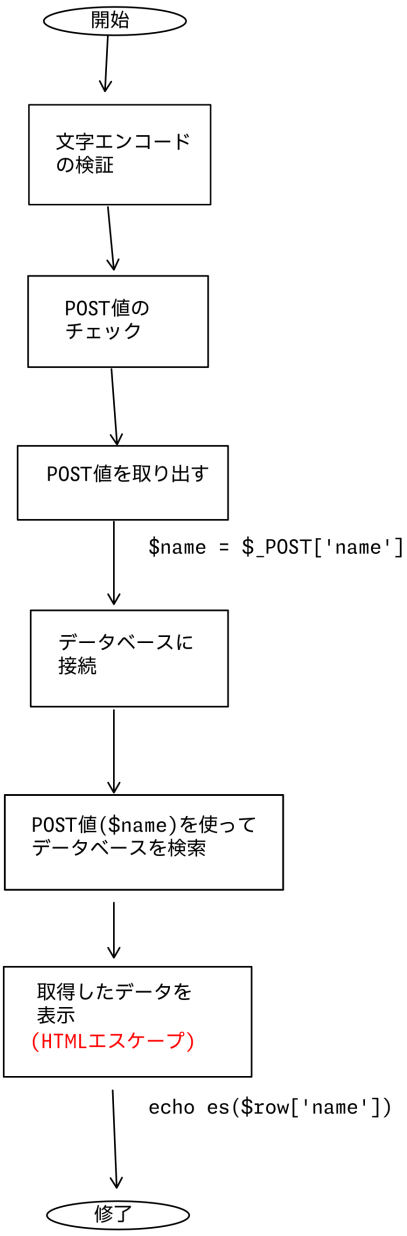
これは、次のような手順になる。

1. 文字エンコードの検証
2. POST 値をセッション変数に渡す
3. セッション変数のチェック
4. セッション変数から値を取り出す
5. 変数の画面出力 (HTML エスケープ)

第 13 章” データベース” のところでも、次のように HTML エスケープは、画面出力のタイミンブで行なわれる。

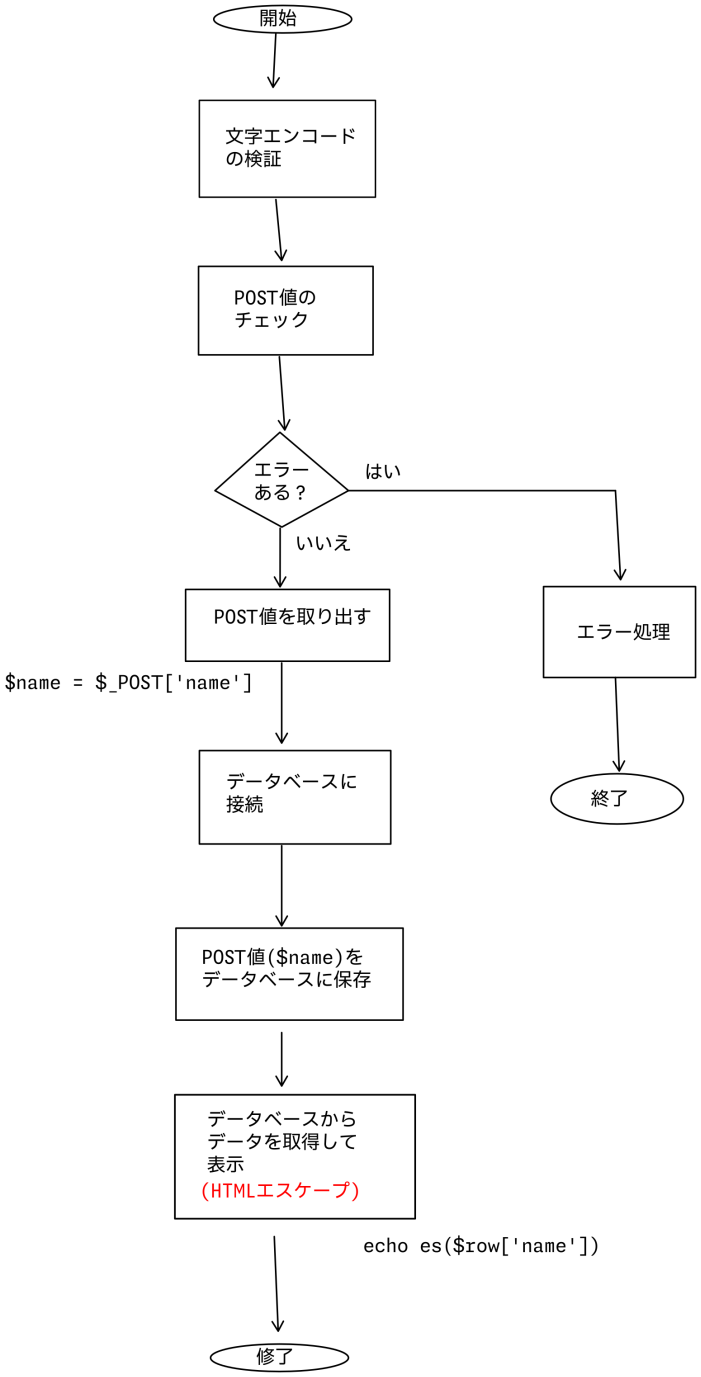
データベースを使う場合

p561 search.php の処理の流れ



データベースを使う場合

p565 insert_member.php の処理の流れ



第 11 章" ファイルの読み書き" でも、同様に、

1. 文字エンコードの検証
2. POST 値をファイルに保存する。
3. ファイルから保存されたデータを変数に読み込む。
4. 変数の画面出力 (HTML エスケープ)

となっている。

つまり、$_POST 値をセッション変数に入れたり、データベース操作に使ったり、ファイルに保存したりする場合は、es() 関数を使わずに操作して、画面出力のタイミングで es() 関数を使っている。

そうではなくて、$_POST 値をほかの変数に入れたとしても、画面出力しかしないのであれば、$_POST をすぐに es() 関数を使って作りなおしている。

そのことの狙いは、おそらく次の 2 点ではなかろうか?

1. HTML エスケープの必要性を強調したい。
   あるいは、HTML エスケープのし忘れがないようにしたい。
2. 画面出力のときに es() 関数を使って記述するのは、じゃまくさい。(初心者には負担になる)

著者に質問したいところだが、残念ながら、ソーテック社の Web ページには本についての質問を受け付けるコーナーがない。
(質問があれば、返信用封筒に返信先を記入し、切手を添付のうえ同封し、封書にて質問してほしいとのことである。)

5.2 HTML エスケープをどのタイミングでおこなうか?

5.2.1 二重エスケープはまずい

エンコード・チェックのあと、すぐに HTML エスケープ をすると、二重エスケープの恐れがある。

次のコードを実行すると、どうなるか?

```
1 $_POST['name'] = "<Taro>";
2 $_POST = es($_POST);      // &lt;Taro&gt;
3 $name = $_POST['name'];
4 echo es($name);           // &amp;lt;Taro\&ampgt;
```

`$_POST['name']` が `"<Taro>"` というデータであった場合、2 行目が実行されたあと、3 行目では `$name` の値は `<Taro>` となる。

4 行目では、それをさらに HTML エスケープしているので、& が `&` に変換されてしまい、`<Taro>` となる。

5.2.2 HTML エスケープは画面出力のタイミングで

php ファイルが 1 つや 2 つなら特に問題にならないだろうけれど、たくさんの php ファイルが連携して 1 つのアプリケーションを構成している場合、ある php ファイルでは`$_POST = es($_POST)`としていて、別のファイルでは、画面出力のタイミングで `echo es($name)` としていたのでは、ソースコードの管理が大変になってくる。

それに、そもそも htmlspecialchars() 関数は、画面出力の際に < などの文字の働きを無効化するのが目的なので、画面出力のタイミングで使うのが望ましい。

5.3 p308 — es() 関数

HTML エスケープを画面出力のタイミンブでのみ行うのであれば、

`$_POST = es($_POST)`

と`$_POST` を丸ごと HTML エスケープする必要はないので、es() 関数は配列を扱う必要がなくなり、p308 の util.php は次のようにシンプルに書ける。

lib_es/lib/util.php

```
function es($str) {
  return htmlspecialchars($str, ENT_QUOTES, "UTF-8");
}
```

次のように 引数で charset を指定できるようにしてもいいが、UTF-8 以外の文字コードを使うこともないので、特に必要はないと思われる。

```
function es($str, $charset="UTF-8") {
  return htmlspecialchars($str, ENT_QUOTES, $charset);
}
```

6　PDO::PARAM_XXX のはたらき

6.1　p559 — PDO::PARAM_XXX

6.1.1　PDO::ATTR_EMULATE_PREPARES, false の場合

PDO::ATTR_EMULATE_PREPARES が true の場合と false の場合で、PDO::PARAM_XXX の働きが違ってくる。ここでは、“false” の場合に限ってまとめてみた。

test テーブル

カラム名	データ型	オプション
id	INT	主キー、自動連番
name	VARCHAR(20)	
age	INT	

このテーブルに次のような SQL 文を実行するとする。

```
$name = "Taro";
$age = 32;

$sql = "INSERT INTO test (name, age) VALUES (:name, :age)";
$stm = $pdo->prepare($sql);
$stm->bindValue(':name', $name);
$stm->bindValue(':age', $age);
$stm->execute();
```

この場合は、PDO::PARAM_XXX の指定をしていないので、$name も $age も 文字列として php から MySQL に値が渡される。この場合、

```
INSERT INTO test (name, age) VALUES ('Taro', '32')
```

が実行される。

数値 32 が文字列として MySQL に渡され、MySQL が数値に変換してカラムに収めている。

これに対して、

```
$stm->bindValue(':name', $name, PDO::PARAM_STR);
$stm->bindValue(':age', $age, PDO::PARAM_INT);
```

と指定した場合は、

```
INSERT INTO test (name, age) VALUES ('Taro', 32)
```

が実行される。

文字列”Taro” は、PARAM_STR の指定があってもなくても変わりはないが、数値 32 は PARAM_INT と指定したほうがよい。数値として MySQL に渡してくれる。

6.1.2　PDO::PARAM_INT の働き

PDO::PARAM_INT の使用には注意が必要である。

PDO::ATTR_EMULATE_PREPARES が false の場合

PDO::PARAM_INT と指定すると、整数も小数点数もそのまま MySQL に渡してくれる。PDO::PARAM_INT は" 整数にする" というはしていない。

PDO::ATTR_EMULATE_PREPARES が true の場合

PDO::PARAM_INT と指定すると、

数値	PDO::PARAM_INT
整数	整数
小数点数	整数

と、PDO::PARAM_INT が働いて、数値を整数にして MySQL に渡すのである。

だから、PDO::ATTR_EMULATE_PREPARES が true の場合、小数点数を MySQL に渡すには PDO::PARAM_STR を使い、文字列として渡し、MySQL の側で float 値に変換して格納してもらうことになる。